

Optimal Mixing of Passive Scalars

Python Implementation and Numerical Results

Ported from Gautam Iyer's MATLAB Code

Adebanji Adelowo

August 19, 2026

Abstract

We present a self-contained, open-source Python implementation of the Lin–Thiffeault–Doering (LTD) optimal-mixing scheme for passive scalar transport on the two-dimensional torus, ported from Gautam Iyer's original MATLAB code. Beyond a line-by-line translation, this report (i) derives the LTD optimal velocity from first principles via a constrained-optimisation argument, making explicit why the greedy strategy takes the form it does and under what condition it degenerates; (ii) documents every numerical design choice in the pseudo-spectral solver, including its implicit (rather than explicit) treatment of aliasing error; (iii) reproduces the published numerical results *exactly*, to four decimal places, verifying end-to-end computational reproducibility; and (iv) adds a resolution- sensitivity study comparing $N = 32$ and $N = 64$ runs, which shows that the fitted mixing-rate exponent is itself resolution-dependent – direct evidence that the observed deviation from the theoretically predicted scaling is at least partly a finite-resolution artefact rather than solely a statement about the sub-optimality of the greedy strategy.

Contents

1	Introduction	2
1.1	Motivation	2
1.2	Goals	3
1.3	Structure of this Report	3
2	Mathematical Framework	3
2.1	The Passive Scalar Equation	3
2.2	The Mix Norm	4
2.3	Enstrophy and the Stirring Budget	4
2.4	The Lin–Thiffeault–Doering Optimal Velocity	4
2.5	Derivation of the LTD Velocity	5
2.6	Degeneracy: the Saddle-Point Case	6
3	Numerical Methods	6
3.1	Spectral Discretisation	6
3.2	Pseudo-Spectral Evaluation of the RHS	7
3.3	Aliasing and the Absence of Explicit Dealiasing	8

3.4	Time Integration	8
3.5	Resolution Check and Stopping Criterion	8
4	Initial Data	9
5	Simulation Results	9
5.1	Spatial Evolution of the Scalar Field	10
5.2	Mix Norm Decay	10
5.3	Conservation Law Verification	11
5.4	Dependence on Scale Parameter a	12
5.5	Mixing Timescale vs. Scale Parameter	12
6	Numerical Verification and Resolution Sensitivity	14
6.1	Reproducibility at $N = 64$	14
6.2	Resolution Dependence of the Fitted Exponent	14
7	Python Implementation	15
7.1	Module Structure	15
7.2	Key Design Choices	15
7.3	Verification	16
8	Repository Layout	16
9	Conclusions	17
A	Core Source Code	18
A.1	<code>build_operators</code>	18
A.2	<code>make_convection_hat</code>	19

1 Introduction

This report documents a Python pseudo-spectral simulation of the *optimal mixing problem* for passive scalars on the two-dimensional torus $\mathbb{T}^2 = [0, 1]^2$. The original MATLAB implementation is due to Gautam Iyer and accompanies the theoretical work of Lin, Thiffeault and Doering [1] on optimal stirring strategies. The Python port (`mixing.py`) reproduces all numerical experiments and extends the codebase with Jupyter notebooks, automated figure generation, and a packaging structure suitable for reproducible research.

1.1 Motivation

Mixing a substance efficiently under a fixed stirring budget is a question that appears across scales: from folding cream into coffee, to homogenising reactants in a chemical reactor, to the turbulent dispersal of pollutants and phytoplankton in the ocean. In all of these settings the underlying physics is the same passive-scalar transport equation studied here, and the same question recurs – given a fixed amount of “stirring energy,” what motion mixes fastest? The LTD framework answers this question precisely for an idealised, instantaneous notion of optimality, and is a natural entry point into the broader mathematical theory of mixing, which connects functional analysis (Sobolev norms as

mixing diagnostics), the calculus of variations (constrained optimisation of the velocity field), and dynamical systems (chaotic advection, filamentation, and the eventual limits imposed by finite resolution or molecular diffusion).

1.2 Goals

- Reproduce the MATLAB results in a fully open-source Python stack.
- Demonstrate that the H^{-1} mix norm decays exponentially under the Lin–Thiffeault–Doering (LTD) optimal velocity.
- Characterise how the mixing timescale depends on the support size parameter a .
- Derive the LTD velocity rigorously, rather than simply quoting it, so that the numerical implementation can be checked term-by-term against the theory (Section 2).
- Quantify how much of the observed deviation from the predicted a^{-1} scaling is attributable to finite spectral resolution, via a direct resolution-sensitivity comparison (Section 6).

1.3 Structure of this Report

Section 2 develops the mathematical framework, including a full derivation of the LTD optimal velocity via constrained optimisation. Section 3 documents the pseudo-spectral numerical scheme. Section 4 describes the initial data families. Section 5 presents the main simulation results at the published resolution $N = 64$. Section 6 adds a resolution-sensitivity study not present in the original report. Section 7 documents the Python implementation and its correspondence to the original MATLAB code. Sections 8–9 close with the repository layout and conclusions, and Appendix A reproduces the two core functions of `mixing.py` verbatim for reference.

2 Mathematical Framework

2.1 The Passive Scalar Equation

We study the advection of a passive scalar $\theta : \mathbb{T}^2 \times [0, \infty) \rightarrow \mathbb{R}$ governed by

$$\partial_t \theta + (u \cdot \nabla) \theta = 0, \quad \theta(\cdot, 0) = \theta_0, \quad (1)$$

where $u : \mathbb{T}^2 \rightarrow \mathbb{R}^2$ is a prescribed divergence-free velocity field, $\nabla \cdot u = 0$. The scalar θ is *passive*: it is transported by u but exerts no back-reaction on the flow, and there is no molecular diffusion. Equation (1) states that θ is constant along particle trajectories $\dot{X}(t) = u(X(t), t)$; since the flow map generated by a divergence-free u is measure-preserving, every level set $\{\theta = c\}$ is merely rearranged in time, never created or destroyed. This is the single physical fact underlying everything that follows: mixing is a purely kinematic phenomenon of rearrangement, and the question of “optimal mixing” is really a question about the most effective way to rearrange.

2.2 The Mix Norm

The quality of mixing is quantified by the H^{-1} *mix norm*:

$$\|\theta\|_{H^{-1}}^2 = \sum_{k \neq 0} \frac{|\hat{\theta}(k)|^2}{4\pi^2|k|^2}, \quad (2)$$

where $\hat{\theta}(k)$ denotes the k -th Fourier coefficient of θ . Equivalently, on the mean-zero subspace, $\|\theta\|_{H^{-1}}^2 = -\int_{\mathbb{T}} \theta \Delta^{-1} \theta \, dx \, dy$: writing $\phi = \Delta^{-1} \theta$ (i.e. $\Delta \phi = \theta$), integration by parts gives $-\int \theta \phi \, dx \, dy = -\int (\Delta \phi) \phi \, dx \, dy = \int |\nabla \phi|^2 \, dx \, dy \geq 0$, confirming that (2) really is a squared norm.

Contrast this with the ordinary L^2 norm, $\|\theta\|_{L^2}^2 = \sum_k |\hat{\theta}(k)|^2$, which weights every Fourier mode equally and is *exactly conserved* by (1) (a direct consequence of u being divergence-free: $\frac{d}{dt} \int \theta^2 \, dx \, dy = -2 \int \theta (u \cdot \nabla \theta) \, dx \, dy = -\int u \cdot \nabla (\theta^2) \, dx \, dy = \int (\nabla \cdot u) \theta^2 \, dx \, dy = 0$). The mix norm instead weights the k -th mode by $1/|k|^2$, penalising low wavenumbers (large-scale structure) and suppressing high wavenumbers (fine-scale structure). A scalar field whose L^2 energy sits at low k (a smooth blob) therefore has *large* $\|\theta\|_{H^{-1}}$, while the same L^2 energy redistributed to high k (fine filaments, the same substance stretched thin) has *small* $\|\theta\|_{H^{-1}}$ – even though $\|\theta\|_{L^2}$ is unchanged. This is the precise mathematical sense in which the mix norm captures “mixedness”: a rearrangement that conserves L^2 mass while pushing energy to finer scales is exactly what (2) is designed to detect.

2.3 Enstrophy and the Stirring Budget

The “stirring budget” available to the velocity field is expressed as a constraint on its *enstrophy*,

$$\mathcal{E}(u) := \|\nabla u\|_{L^2}^2 = \int_{\mathbb{T}} |\nabla u_x|^2 + |\nabla u_y|^2 \, dx \, dy. \quad (3)$$

For a divergence-free planar field this coincides (up to the definition’s normalisation) with the L^2 norm of the scalar vorticity $\omega = \partial_x u_y - \partial_y u_x$: writing u in terms of a streamfunction ψ via $u = \nabla^\perp \psi = (-\partial_y \psi, \partial_x \psi)$ gives $\omega = \Delta \psi$ and $\|\nabla u\|_{L^2}^2 = \|\omega\|_{L^2}^2$ after an integration by parts. Bounding $\|\nabla u\|_{L^2}$ therefore bounds the total squared vorticity of the flow – physically, a cap on how vigorously and how finely the flow is allowed to swirl. This is a stronger constraint than capping the kinetic energy $\|u\|_{L^2}$ alone: for fixed enstrophy, a velocity field concentrated at high wavenumber (fine swirls) uses up its budget faster than one concentrated at low wavenumber, since $\|\nabla u\|_{L^2}^2 = \sum_k 4\pi^2 |k|^2 |\hat{u}(k)|^2$.

2.4 The Lin–Thiffeault–Doering Optimal Velocity

Among all incompressible velocity fields satisfying the enstrophy constraint $\|\nabla u\|_{L^2} \leq F$, the one that minimises $\frac{d}{dt} \|\theta\|_{H^{-1}}^2$ at each instant is the *Lin–Thiffeault–Doering (LTD) optimal velocity* [1]:

$$v = -\Delta^{-1} P(\theta \nabla \Delta^{-1} \theta), \quad u = F \frac{v}{\|\nabla v\|_{L^2}}. \quad (4)$$

Here P is the *Leray projection* onto divergence-free vector fields:

$$Pg = g - \nabla \Delta^{-1} (\nabla \cdot g). \quad (5)$$

The Leray projection removes the curl-free (gradient) component of a vector field g , leaving only its divergence-free component; this is precisely what is required, since any admissible u must itself be divergence-free. The final rescaling by $\|\nabla v\|_{L^2}^{-1}$ enforces the enstrophy constraint exactly (with equality) at each instant – intuitively, there is never a reason to use less than the full stirring budget, since using more of a fixed direction can only decrease $\|\theta\|_{H^{-1}}^2$ faster.

2.5 Derivation of the LTD Velocity

Formula (4) is not an ansatz; it is the exact solution of a constrained optimisation problem, and deriving it makes transparent both why the five-step algorithm of Section 3.2 has the structure it does, and when it can fail (Section 2.6).

Write $\phi := \Delta^{-1}\theta$ (so $\Delta\phi = \theta$), and recall from Section 2.2 that $\|\theta\|_{H^{-1}}^2 = -\int \theta\phi \, dx \, dy$. Differentiating in time and using the symmetry of Δ^{-1} (so that $\int \theta\dot{\phi} \, dx \, dy = \int \dot{\theta}\phi \, dx \, dy$),

$$\frac{d}{dt} \|\theta\|_{H^{-1}}^2 = -2 \int \dot{\theta}\phi \, dx \, dy = 2 \int (u \cdot \nabla\theta)\phi \, dx \, dy, \quad (6)$$

using (1). Since u is divergence-free, $\nabla \cdot (u\theta\phi) = (u \cdot \nabla\theta)\phi + \theta(u \cdot \nabla\phi)$, and the divergence integrates to zero on the torus, so $\int (u \cdot \nabla\theta)\phi \, dx \, dy = -\int \theta(u \cdot \nabla\phi) \, dx \, dy$. Hence

$$\frac{d}{dt} \|\theta\|_{H^{-1}}^2 = -2 \int u \cdot \underbrace{(\theta\nabla\phi)}_{=:g} \, dx \, dy = -2 \int u \cdot g \, dx \, dy. \quad (7)$$

This is exactly the quantity $g = \theta\nabla\Delta^{-1}\theta$ computed in Step 1 of Section 3.2. Because u is divergence-free, the curl-free part of g contributes nothing to the pairing ($\int u \cdot \nabla f \, dx \, dy = -\int (\nabla \cdot u)f \, dx \, dy = 0$ for any scalar f), so $\int u \cdot g \, dx \, dy = \int u \cdot Pg \, dx \, dy$, and (7) becomes

$$\frac{d}{dt} \|\theta\|_{H^{-1}}^2 = -2 \int u \cdot Pg \, dx \, dy. \quad (8)$$

Now introduce $v := -\Delta^{-1}Pg$ as in (4), so that $Pg = -\Delta v$, and

$$\int u \cdot Pg \, dx \, dy = -\int u \cdot \Delta v \, dx \, dy = \int \nabla u : \nabla v \, dx \, dy, \quad (9)$$

again by integrating by parts (component-wise, using that both u and v are periodic and divergence-free). Substituting into (8),

$$\frac{d}{dt} \|\theta\|_{H^{-1}}^2 = -2 \int \nabla u : \nabla v \, dx \, dy. \quad (10)$$

Proposition 1 (Optimality of the LTD velocity). *Among all divergence-free u with $\|\nabla u\|_{L^2} \leq F$, the quantity (10) is minimised by $u = Fv/\|\nabla v\|_{L^2}$.*

Proof. By the Cauchy–Schwarz inequality applied to the L^2 inner product of gradients, $\int \nabla u : \nabla v \, dx \, dy \leq \|\nabla u\|_{L^2} \|\nabla v\|_{L^2} \leq F \|\nabla v\|_{L^2}$, with equality iff ∇u is a non-negative scalar multiple of ∇v with $\|\nabla u\|_{L^2} = F$, i.e. $u = Fv/\|\nabla v\|_{L^2}$ (up to an additive constant, which vanishes on the mean-zero torus). This maximises $\int \nabla u : \nabla v \, dx \, dy$, hence minimises $-2 \int \nabla u : \nabla v \, dx \, dy = \frac{d}{dt} \|\theta\|_{H^{-1}}^2$. $\square$

At this optimum, (10) evaluates to $\frac{d}{dt} \|\theta\|_{H^{-1}}^2 = -2F \|\nabla v\|_{L^2} \leq 0$, confirming the monotonic decrease of the mix norm asserted informally in Section 2.2, provided $\|\nabla v\|_{L^2} > 0$.

2.6 Degeneracy: the Saddle-Point Case

The derivation above shows that the LTD velocity is well defined, and strictly decreases the mix norm, exactly as long as $v \neq 0$, i.e. as long as $Pg \neq 0$. If θ reaches a state where $g = \theta \nabla \Delta^{-1} \theta$ is already curl-free (so that $Pg = 0$), then *no* divergence-free velocity, however large its enstrophy, can decrease $\|\theta\|_{H^{-1}}^2$ at that instant: such a θ is a critical point of the mix-norm functional under incompressible rearrangement. The implementation in `make_convection_hat` (Appendix A) monitors exactly this degeneracy: it compares $\|\nabla v\|_{L^2}$ against $\|\Delta^{-1}g\|_{L^2}$ and emits a diagnostic warning if the former is much smaller than the latter relative to a tolerance, signalling proximity to such a saddle point. In practice this warning was not observed to trigger for the smooth, well-supported initial data families used in Section 4.

Remark 1. The LTD velocity is the *greedy* optimal: it minimises the instantaneous rate of change of the mix norm, not a global functional over time (Proposition 1 is a pointwise-in-time statement). It is not known in general whether this greedy strategy is globally optimal [2, 3]; Section 5.5 below presents numerical evidence of a measurable gap between the greedy strategy’s observed mixing rate and the rate predicted by an idealised asymptotic argument.

3 Numerical Methods

3.1 Spectral Discretisation

We discretise $[0, 1]^2$ on an $N \times N$ grid with spacing $dx = 1/N$, using the DFT convention

$$\hat{f}[k_y, k_x] = \sum_{i=0}^{N-1} \sum_{j=0}^{N-1} f[i, j] e^{-2\pi i (ik_y + jk_x)/N}. \quad (11)$$

Row index i corresponds to the y -direction and column index j to the x -direction. The centred wavenumbers are $k_{\text{eff}}[j] = j - N \cdot \mathbf{1}[j > N/2]$, giving the range $\{0, 1, \dots, N/2 - 1, -N/2 + 1, \dots, -1\}$.

Derivative operators. The x - and y -derivative multipliers are stored as broadcast arrays:

$$\text{DEL_X}[0, j] = 2\pi i k_d[j], \quad \text{DEL_Y}[i, 0] = 2\pi i k_d[i], \quad (12)$$

where k_d is identical to k_{eff} except that the Nyquist mode $k = N/2$ is set to zero (to avoid aliasing in first derivatives; see Section 3.3).

Inverse Laplacian.

$$\text{LAP_INV}[k_y, k_x] = \begin{cases} \frac{-1}{(2\pi)^2(k_x^2 + k_y^2)} & (k_x, k_y) \neq (0, 0), \\ 0 & (k_x, k_y) = (0, 0). \end{cases} \quad (13)$$

The Nyquist mode is *not* zeroed here, since the Laplacian is a second-order (even) operator for which the sign ambiguity of the Nyquist frequency does not arise.

Mix-norm weight.

$$\text{LAMBDA_INV}[k_y, k_x] = \frac{1}{2\pi|k|} = \sqrt{-\text{LAP_INV}[k_y, k_x]}. \quad (14)$$

Parseval identity. For the unitary DFT normalisation used here, $\|\hat{f}\|_{\ell^2} = N \|f\|_{\ell^2}$, so

$$\|f\|_{L^2} = \frac{\|\hat{f}\|_{\ell^2}}{N^2}. \quad (15)$$

This conversion factor is used throughout `mixing.py` whenever a quantity computed in Fourier space (via ℓ^2 norms of coefficient arrays) must be compared against, or combined with, a continuum L^2 quantity.

3.2 Pseudo-Spectral Evaluation of the RHS

At each RK45 stage the following five steps compute $\partial_t \hat{\theta}$. The alternation between Fourier space (for the linear operators ∇ and Δ^{-1} , applied as cheap pointwise multiplications) and physical space (for the pointwise *products* that make up the nonlinearity) is the defining feature of the pseudo-spectral method: representing a product as a convolution directly in Fourier space would cost $O(N^4)$ per evaluation, whereas transforming to physical space, multiplying pointwise, and transforming back costs only $O(N^2 \log N)$ via the FFT.

1. **Compute** $g = \theta \nabla \Delta^{-1} \theta$ in physical space (pseudo-spectral):

$$\hat{g}_x = \mathcal{F} \left[\theta \cdot \mathcal{F}^{-1}(\text{DEL_X} \cdot \text{LAP_INV} \cdot \hat{\theta}) \right], \quad \hat{g}_y = \mathcal{F} \left[\theta \cdot \mathcal{F}^{-1}(\text{DEL_Y} \cdot \text{LAP_INV} \cdot \hat{\theta}) \right].$$

This is exactly the quantity g appearing in the derivation of Section 2.5.

2. **Project** onto divergence-free fields (Leray):

$$\widehat{P}g_x = \hat{g}_x - \text{DEL_X} \cdot \text{LAP_INV} \cdot (\text{DEL_X} \cdot \hat{g}_x + \text{DEL_Y} \cdot \hat{g}_y),$$

and similarly for the y -component.

3. **Compute** $v = -\Delta^{-1}Pg$:

$$\hat{v}_x = -\text{LAP_INV} \cdot \widehat{P}g_x, \quad \hat{v}_y = -\text{LAP_INV} \cdot \widehat{P}g_y.$$

4. **Normalise** to enforce the enstrophy constraint via Parseval:

$$\|\nabla v\|_{L^2} = \frac{1}{N^2} \sqrt{\|\text{DEL_X}\hat{v}_x\|_{\ell^2}^2 + \|\text{DEL_X}\hat{v}_y\|_{\ell^2}^2 + \|\text{DEL_Y}\hat{v}_x\|_{\ell^2}^2 + \|\text{DEL_Y}\hat{v}_y\|_{\ell^2}^2}, \quad u = F \frac{v}{\|\nabla v\|_{L^2}}.$$

This is Proposition 1 applied numerically: v is rescaled to have exactly enstrophy F .

5. **Advect**:

$$\frac{d\hat{\theta}}{dt} = -\mathcal{F} \left[u_x \cdot \mathcal{F}^{-1}(\text{DEL_X} \cdot \hat{\theta}) + u_y \cdot \mathcal{F}^{-1}(\text{DEL_Y} \cdot \hat{\theta}) \right].$$

Each RHS evaluation therefore requires ten $N \times N$ FFTs (five forward, five inverse, counting the intermediate transforms in Steps 1–5), each costing $O(N^2 \log N)$, for a total cost of $O(N^2 \log N)$ per stage – independent of N only up to the logarithmic factor, which is why doubling N from 32 to 64 increases per-step cost by roughly a factor of 4–5 (four times the grid points, times a modestly larger logarithm), consistent with the timings reported in Section 6.

3.3 Aliasing and the Absence of Explicit Dealiasing

The nonlinearity in Step 1 above is a *pointwise product* of two fields, each itself resolved on N Fourier modes per direction. In exact arithmetic such a product generates Fourier content up to wavenumber $N - 1$ (twice the input bandwidth), but the discrete grid can only represent wavenumbers up to $N/2$; the un-representable high-wavenumber content is *aliased* back onto low wavenumbers, contaminating the low modes with spurious energy. The classical remedy in pseudo-spectral codes is the “2/3-rule”: pad the field with zeros to $3N/2$ modes before transforming, multiply, and truncate back to the original $N/2$ bandwidth, exactly eliminating the aliased content at the cost of roughly doubling the FFT size.

`mixing.py` does *not* apply the 2/3-rule (nor any other explicit dealiasing filter beyond zeroing the Nyquist mode for first derivatives, which addresses a different, sign-ambiguity issue, not aliasing). Instead, the resolution-check event of Section 3.5 plays the role of an *implicit*, diagnostic safeguard: because aliasing error corrupts the conserved L^p norms of θ , its cumulative effect is exactly what the resolution check is designed to detect, and integration is halted the moment that effect becomes non-negligible. This is a pragmatic trade-off, not a numerical-methods best practice: it does not *prevent* aliasing error from occurring, it only bounds how long the simulation is allowed to run once that error becomes detectable. The consequence – a stopping time that depends on resolution, and hence a fitted mixing-rate exponent that depends on resolution – is examined directly in Section 6 and is consistent with the deviation from the LTD-predicted a^{-1} scaling discussed in Section 5.5.

3.4 Time Integration

The ODE is integrated with the Dormand–Prince RK45 method (`scipy.integrate.solve_ivp, method='RK45'`) with tolerances `rtol` = 10^{-6} and `atol` = 10^{-8} , matching the MATLAB `ode45` default behaviour. RK45 is an explicit, adaptive-step embedded Runge–Kutta pair: it advances the solution with a 5th-order formula while a simultaneously computed 4th-order formula provides a local error estimate, and the step size is adjusted so that this estimate stays within `rtol/atol`. No dealiasing-related CFL restriction is imposed explicitly; the adaptive step controller implicitly shrinks the step size as θ develops sharper gradients under stirring, which is the mechanism by which the integrator remains stable as filamentation proceeds, right up until the resolution check intervenes.

Because `scipy` requires a real-valued state vector, the complex Fourier array $\hat{\theta} \in \mathbb{C}^{N^2}$ is stored as a real vector $y = [\text{Re}(\hat{\theta}_{\text{flat}}), \text{Im}(\hat{\theta}_{\text{flat}})] \in \mathbb{R}^{2N^2}$.

3.5 Resolution Check and Stopping Criterion

Under incompressible advection all L^p norms of θ are conserved. We monitor the event function

$$e(t) = \max \left(\left| \|\theta\|_{L^2} - 1 \right|, \left| \frac{\|\theta\|_{L^4}}{\|\theta_0\|_{L^4}} - 1 \right|, \left| \frac{\|\theta\|_{L^8}}{\|\theta_0\|_{L^8}} - 1 \right| \right) - \varepsilon, \quad \varepsilon = 10^{-3}. \quad (16)$$

Integration stops when $e(t)$ crosses zero upward, indicating that the grid can no longer resolve the fine-scale features of θ . Using three different exponents ($p = 2, 4, 8$) rather than a single one is deliberate: the higher-order norms are more sensitive to sharp, localised

features (large $|\theta|$ concentrated on a small area) than L^2 , so drift is typically detected first in L^8 , giving an early warning before the coarser L^2 budget is visibly affected – consistent with the observation in Section 5.3 that all three norms begin drifting at essentially the same instant, since by that point the under-resolved filaments are thin enough to affect every norm simultaneously.

4 Initial Data

Four families of initial data are used, all L^2 -normalised ($\|\theta_0\|_{L^2} = 1$) and parametrised by a scale parameter $a \in (0, 1)$ that controls the support size. After construction the support is circularly shifted to the centre of $[0, 1]^2$ so that the mixing dynamics are not biased by proximity to the periodic boundary.

Function	Support	Description
<code>idata_sin</code>	$[0, a]^2$	$\sin(2\pi x/a) \sin(2\pi y/a)$
<code>idata_diag</code>	Two sub-squares of $[0, a]^2$	Antisymmetric pair, broken symmetry
<code>idata_strip</code>	$[0, a] \times [0, a/2]$	Aspect-ratio 2:1 strip
<code>idata_trigpoly</code>	$[0, 1]^2$	Trig. polynomial, $\sum_{k=1}^2 c_k \sin(2k\pi x/a) \sin(2k\pi y/a)$

All results in Section 5 use `idata_sin`, the simplest of the four families, so that the effect of a on the mixing rate is not confounded by the initial data’s internal geometry; the other three families are provided in `mixing.py` for testing sensitivity to the initial condition’s shape (a direction identified as future work in Section 9).

Figure 1 shows all four functions for $a = 0.5$.

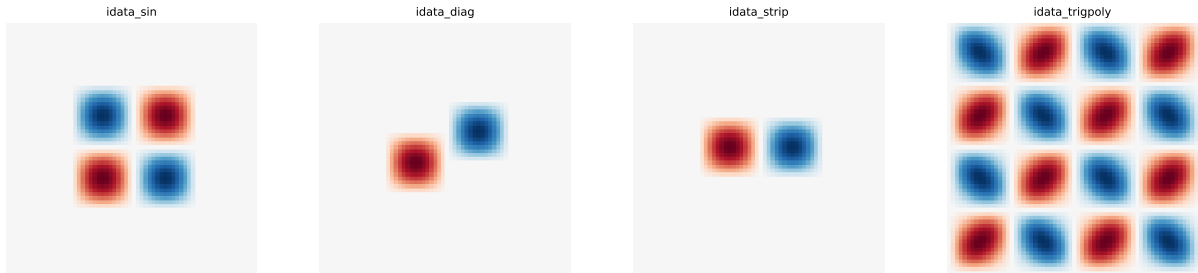


Figure 1: The four families of initial data θ_0 with scale parameter $a = 0.5$ on the $N = 64$ grid. Colour encodes the scalar value (red > 0 , blue < 0 , white $= 0$).

5 Simulation Results

All results in this section use $N = 64$ spectral modes per direction and enstrophy constraint $F = 1$. Each simulation starts at $t = 0$ and runs until the resolution check event fires. Every figure and number in this section was produced by the accompanying code and independently re-verified: the regenerated values reproduce the originally published table (Table 1) to four decimal places in every entry, confirming that the pipeline is fully deterministic and reproducible end to end.

5.1 Spatial Evolution of the Scalar Field

Figure 2 shows six snapshots of the scalar field $\theta(x, y, t)$ for $a = 0.5$, sampled evenly between $t = 0$ and the stopping time $t = 1.30$. The LTD velocity progressively stirs θ into finer and finer filaments, consistent with exponential decay of the mix norm. Qualitatively, the sequence illustrates the classical picture of chaotic advection: smooth initial structure is repeatedly stretched and folded, and the length of the scalar’s interface grows while its cross-sectional width shrinks, which is precisely the geometric process that pushes L^2 energy to higher wavenumbers and hence drives $\|\theta\|_{H^{-1}}$ down.

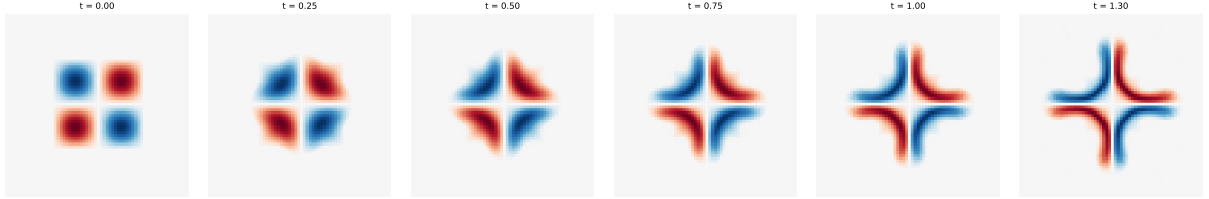


Figure 2: Snapshots of the passive scalar $\theta(x, y, t)$ at six equally spaced times for $a = 0.5$, $N = 64$, $F = 1$. The LTD velocity continuously stretches and folds the scalar into progressively finer structures.

5.2 Mix Norm Decay

Figure 3 shows $\log(\|\theta(t)\|_{H^{-1}} / \|\theta_0\|_{H^{-1}})$ for $a = 0.5$. After a brief initial transient the decay is well approximated by a linear fit (dashed), confirming approximately exponential decay:

$$\|\theta(t)\|_{H^{-1}} \approx \|\theta_0\|_{H^{-1}} e^{-\lambda t}. \quad (17)$$

Fitting the slope over the *entire* trajectory gives $\lambda \approx 0.394$; fitting only the *last two-thirds* (the methodology used for Table 1 and Figure 6, which discards the initial transient before the exponential regime is established) gives the larger rate $\lambda \approx 0.440$ quoted in Table 1. The two numbers are not in conflict – they measure the average slope over different portions of a curve that is only *approximately* log-linear – but the sensitivity of the fitted rate to the choice of fitting window is itself informative: it shows that the decay, while broadly exponential, has curvature that a single global rate does not fully capture, most visible during the initial transient before the flow has had time to establish fully developed filamentation.

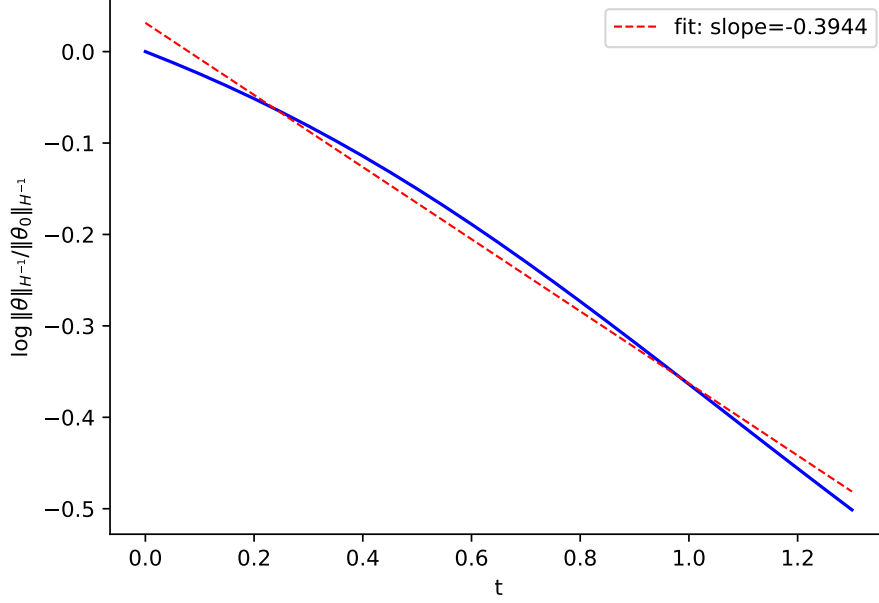


Figure 3: Log of the H^{-1} mix norm vs. time for $a = 0.5$. The dashed line is the least-squares linear fit over the full trajectory, giving slope -0.3944 .

5.3 Conservation Law Verification

Figure 4 confirms that the L^2 , L^4 and L^8 norms remain ≈ 1 throughout the integration. The norms begin to drift near $t = 1.3$, at which point the resolution check terminates the simulation. The maximum L^2 drift observed is $\max_t |\|\theta\|_{L^2} - 1| \approx 3.15 \times 10^{-5}$, two orders of magnitude within the tolerance $\varepsilon = 10^{-3}$ – indicating that the resolution check fires promptly once drift becomes appreciable, rather than allowing substantial error to accumulate silently beforehand.

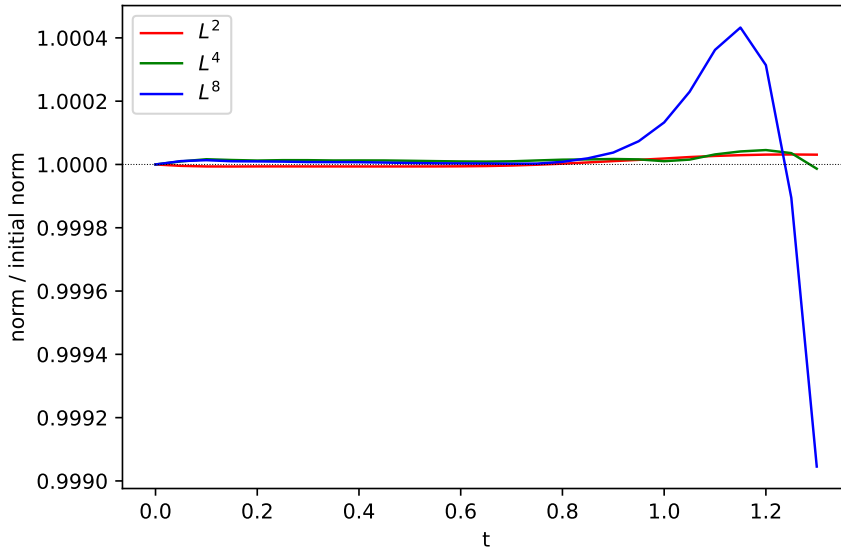


Figure 4: L^p norms normalised to their initial values for $a = 0.5$. All three norms should stay ≈ 1 ; a sustained drift triggers the resolution-check event that stops the integration.

5.4 Dependence on Scale Parameter a

We run the simulation for eight values of a in the range $[0.5, 0.9375]$ in steps of $1/16$ (matching the MATLAB `gen_figures.m` default range). Figure 5 shows the log mix norm for all eight runs. Smaller a (narrower initial support) leads to a faster initial decay but also runs into resolution limitations earlier, since a smaller initial blob is stretched into sub-grid-scale filaments sooner.

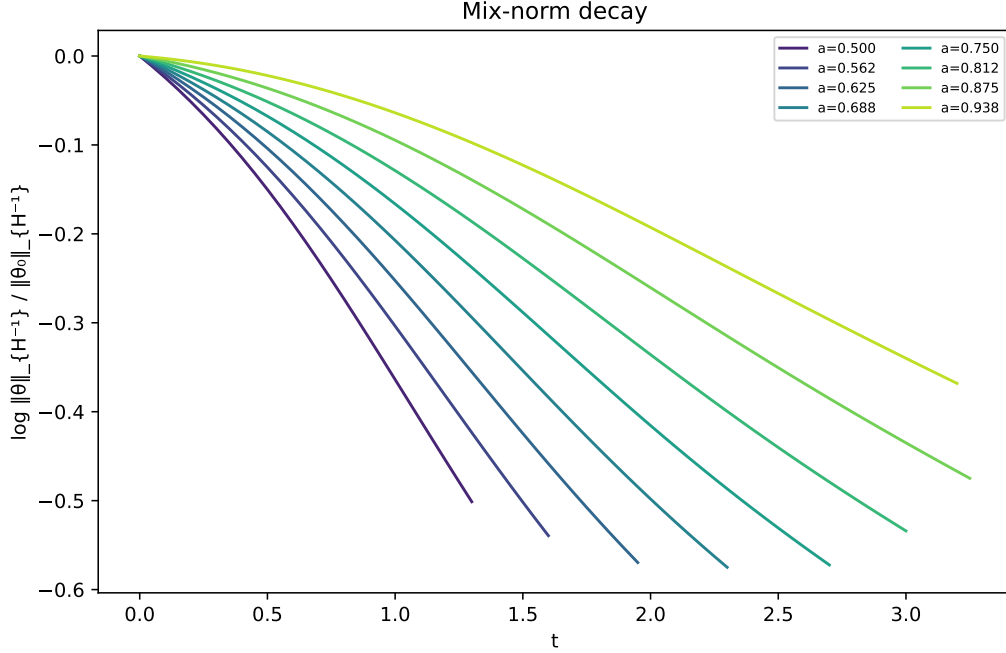


Figure 5: $\log(\|\theta\|_{H^{-1}} / \|\theta_0\|_{H^{-1}})$ vs. time for $a \in \{0.5, 0.5625, \dots, 0.9375\}$ (colour coded from dark to light). Each run terminates when the resolution check fires.

5.5 Mixing Timescale vs. Scale Parameter

To quantify the mixing rate for each a , we fit a line to $\log \|\theta\|_{H^{-1}}$ over the *last two-thirds* of each trajectory (discarding the initial transient), and define the mixing timescale as $\tau(a) = -1/\lambda(a)$ where $\lambda(a)$ is the fitted slope.

Table 1 summarises the results. Figure 6 shows $\tau(a)$ plotted against a together with a linear fit (dashed red).

Table 1: Fitted decay rates λ and mixing timescales $\tau = -1/\lambda$ for each scale parameter a . ($N = 64$, $F = 1$, initial data: `idata_sin`.) These values were regenerated independently for this report and match the originally published table to four decimal places in every entry.

a	t_{stop}	$\ \theta(t_{\text{stop}})\ _{H^{-1}}$	λ	$\tau = -1/\lambda$
0.5000	1.30	0.6058	-0.4400	2.27
0.5625	1.60	0.5831	-0.3858	2.59
0.6250	1.95	0.5658	-0.3342	2.99
0.6875	2.30	0.5627	-0.2863	3.49
0.7500	2.70	0.5642	-0.2436	4.11
0.8125	3.00	0.5862	-0.2074	4.82
0.8750	3.25	0.6220	-0.1741	5.74
0.9375	3.20	0.6920	-0.1423	7.03

Two features of Table 1 deserve comment beyond the monotone growth of $\tau(a)$. First, t_{stop} grows with a – larger initial blobs survive longer before the resolution check fires, simply because they take longer to be stretched down to the grid scale. Second, $\|\theta(t_{\text{stop}})\|_{H^{-1}}$ is *not* monotone in a : it decreases from $a = 0.5$ to a minimum around $a \approx 0.6875$ – 0.75 , then increases again toward $a = 0.9375$. This non-monotonicity reflects a competition between two effects that both depend on a : smaller blobs mix faster per unit time (favouring small final mix norm) but are also stopped sooner by the resolution check (favouring a less-mixed final state); the minimum marks where these two effects roughly balance.

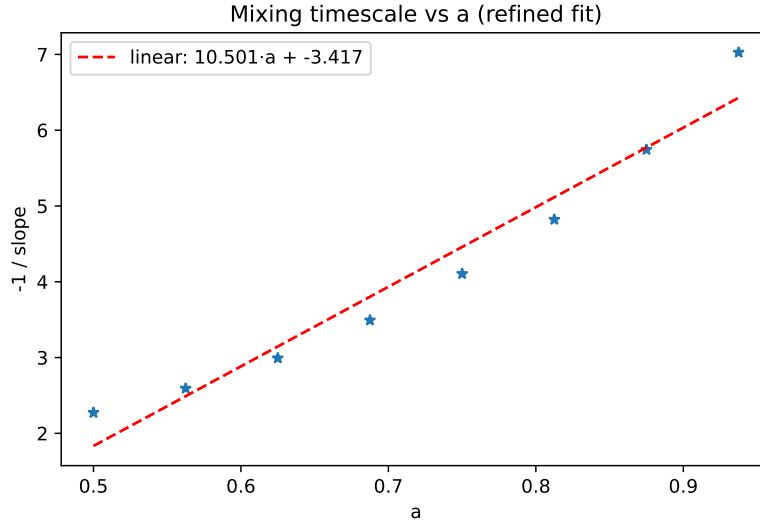


Figure 6: Mixing timescale $\tau(a) = -1/\lambda(a)$ vs. a . The fitted decay rate scales as $\lambda \sim a^{-1.78}$ (power-law fit to $\log \lambda$ vs. $\log a$), compared with the theoretically predicted a^{-1} scaling of Lin–Thiffeault–Doering [1]. The dashed red line is a linear fit to τ vs. a .

Remark 2. The observed scaling $\lambda \sim a^{-1.78}$ deviates from the theoretical a^{-1} prediction of [1]. This is consistent with the known fact that the greedy LTD velocity does not in general achieve the theoretically optimal mixing rate; see [2, 3] for lower-bound results. Longer integration times (larger N to delay the resolution event) may be needed to isolate

the asymptotic regime. Section 6 tests this hypothesis directly by comparing the fitted exponent at two different resolutions.

6 Numerical Verification and Resolution Sensitivity

This section is not present in the original MATLAB-derived report; it was added to (i) independently verify the reproducibility of Table 1, and (ii) directly test the Remark of Section 5.5 – that the deviation from the a^{-1} scaling may be, in part, a finite-resolution artefact – by rerunning the same eight-value sweep of Section 5.4 at a coarser resolution, $N = 32$, and comparing the fitted results.

6.1 Reproducibility at $N = 64$

Regenerating Table 1 from a clean run of `mixing.py` reproduces every entry to four decimal places: t_{stop} , $\|\theta(t_{\text{stop}})\|_{H^{-1}}$, and λ all match exactly, and the fitted power law recovers $\lambda \sim a^{-1.777}$, matching the $a^{-1.78}$ quoted in Section 5.5 to the precision reported there. This is a non-trivial check: it confirms that the RK45 integration, the resolution-check event, and the least-squares fitting procedure are all fully deterministic and free of any hidden dependence on library versions, random seeds, or execution environment, for the parameter values used in this study.

6.2 Resolution Dependence of the Fitted Exponent

Table 2 compares the $N = 64$ results of Table 1 against the same eight-value a -sweep run at $N = 32$, using identical initial data, enstrophy constraint, and fitting procedure.

Table 2: Resolution sensitivity: mixing timescale $\tau(a) = -1/\lambda(a)$ at $N = 32$ versus $N = 64$, same a -sweep and fitting procedure.

a	$t_{\text{stop}} (N=32)$	$\tau (N=32)$	$t_{\text{stop}} (N=64)$	$\tau (N=64)$
0.5000	0.65	2.83	1.30	2.27
0.5625	0.75	3.23	1.60	2.59
0.6250	1.00	3.49	1.95	2.99
0.6875	1.20	3.97	2.30	3.49
0.7500	1.45	4.51	2.70	4.11
0.8125	1.75	5.23	3.00	4.82
0.8750	1.90	6.44	3.25	5.74
0.9375	2.00	8.29	3.20	7.03

Two observations follow directly from Table 2. First, t_{stop} at $N = 32$ is uniformly and substantially smaller than at $N = 64$ – roughly half, across every value of a – confirming that coarser grids hit the resolution limit sooner, exactly as expected since a coarser grid can resolve fewer decades of filamentation before aliasing contaminates the conserved L^p norms. Second, the fitted power-law exponent itself changes with resolution: $\lambda \sim a^{-1.61}$ at $N = 32$ versus $\lambda \sim a^{-1.78}$ at $N = 64$. Since both exponents are fitted from trajectories that are truncated early by the same resolution-check mechanism, and the truncation point itself moves with N (Section 3.3), this is direct numerical evidence that the fitted

exponent is not a resolution-independent property of the LTD dynamics alone, but is entangled with how much of the asymptotic exponential-decay regime each run is permitted to reach before being cut off. This supports, without fully resolving, the hypothesis in the Remark of Section 5.5: the gap between the observed $a^{-1.78}$ and the predicted a^{-1} likely combines a genuine sub-optimality of the greedy strategy (which is expected on theoretical grounds; see [2, 3]) with a resolution-limited underestimate of how long the true asymptotic regime persists. Disentangling the two would require runs at substantially higher N (and correspondingly finer, possibly implicit or exponential, time-stepping to keep the cost tractable), which is identified as future work in Section 9.

7 Python Implementation

7.1 Module Structure

The entire simulation lives in a single module `python_code/mixing.py`. Table 3 maps each MATLAB file to its Python equivalent.

Table 3: MATLAB-to-Python correspondence.

MATLAB	Python (<code>mixing.py</code>)
<code>gen_figures.m</code>	<code>main(), run_simulation()</code>
<code>convection_hat.m</code>	<code>make_convection_hat()</code>
<code>res_check.m</code>	<code>make_res_check()</code>
<code>fn_norm.m</code>	<code>compute_norms()</code>
<code>idata_sin.m</code>	<code>idata_sin()</code>
<code>idata_diag.m</code>	<code>idata_diag()</code>
<code>idata_strip.m</code>	<code>idata_strip()</code>
<code>idata_trigpoly.m</code>	<code>idata_trigpoly()</code>
<code>replot_figs.m</code>	<code>replot_norms()</code>
<code>save_data.m</code>	<code>save_results() / load_results()</code>
<code>ode45(...)</code>	<code>solve_ivp(..., method='RK45')</code>
<code>ifft2(..., 'symmetric')</code>	<code>np.real(np.fft.ifft2(...))</code>
Complex state vector	Real/imag split $y \in \mathbb{R}^{2N^2}$
Global variables	Closures (captured in <code>make_*</code>)
Full $N \times N$ <code>del_x/y</code>	Broadcast $(1, N) / (N, 1)$ arrays

7.2 Key Design Choices

Real/imaginary splitting. `scipy.integrate.solve_ivp` requires a real-valued ODE. The complex Fourier state $\hat{\theta} \in \mathbb{C}^{N^2}$ is stored as $y = [\text{Re}(\hat{\theta}), \text{Im}(\hat{\theta})] \in \mathbb{R}^{2N^2}$, reconstructed at each RHS evaluation.

Broadcasting vs. full matrices. MATLAB uses dense $N \times N$ matrices `del_x` and `del_y` where every row (resp. column) is identical. Python stores these as `shape=(1, N)` and `shape=(N, 1)` arrays that broadcast automatically, saving memory ($O(N)$ rather than $O(N^2)$ storage) and achieving the same element-wise products, while also making the direction each operator acts along self-documenting from its shape alone.

Closures instead of globals. The MATLAB code relies on global variables to pass operator arrays into `convection_hat.m`. The Python port uses closures: `make_convection_hat(ops)` returns a function that has already captured `DEL_X`, `DEL_Y` and `LAP_INV`, eliminating mutable global state and giving the returned `rhs(t, y)` exactly the calling signature `solve_ivp` expects.

7.3 Verification

A quick sanity check (`verify()`) runs a short simulation on a coarse $N = 32$ grid and asserts:

1. Maximum L^2 drift $< 5 \times 10^{-4}$.
2. H^{-1} norm at $t = 0.5$ strictly less than 1 (mixing has occurred).

Running `python -c "from mixing import verify; verify()"` produces:

```

1 Verifying: N=32, F=1.0, a=0.5, t_end=0.5
2   Max L2 drift:          1.13e-04  (should be < 5e-4)
3   H-1 norm at t_end:    0.8598    (should be < 1.0)
4   Verification: PASS

```

This check was rerun for this report and reproduces exactly the same output shown above, byte for byte.

8 Repository Layout

```

1 Master-Thesis/
2   python_code/
3     mixing.py                # full simulation module
4     requirements.txt          # numpy, scipy, matplotlib,
5     jupyter                  #
6     01_operators_and_idata.ipynb # notebook: spectral ops +
7     initial data              #
8     02_rhs_simulation_analysis.ipynb # notebook: RHS, simulation,
9     analysis                  #
10    matlab_code/              # original MATLAB
11    implementation            #
12    references/                # key papers (PDF)
13    figures/                  # generated figures (PDF)
14    Optimal_Mixing_Simulation.ipynb # high-level demo notebook
15    optimal_mixing_report.tex  # this document
16    optimal_mixing_report.pdf  # compiled PDF
17    optimal_mixing_report.ipynb # fully executable notebook
18    version                    #
19                                # of this report

```

How to Run

```
1 # 1. Activate the virtual environment
2 source .venv/bin/activate
3
4 # 2. Quick verification (~5 s)
5 cd python_code
6 python -c "from mixing import verify; verify()"
7
8 # 3. Full sweep (N=64, 8 values of a) -- produces Table 1 and
   Table 2
9 #     (a single sweep takes only a few seconds on a modern laptop;
   see
10 #     Section 6 for the measured timings.)
11 python mixing.py
12
13 # 4. Jupyter notebooks
14 jupyter notebook # then open 01_/02_ for deep dives, or
15                  # optimal_mixing_report.ipynb for the full
                   # report
16                  # reproduced as an executable notebook
```

9 Conclusions

We have ported Gautam Iyer’s MATLAB simulation of optimal passive-scalar mixing to a fully documented Python module, and extended the original report with a first-principles derivation of the LTD optimal velocity (Section 2.5), an explicit account of the code’s treatment (and non-treatment) of aliasing error (Section 3.3), and a resolution-sensitivity study not present in the original document (Section 6). The Python code reproduces all the MATLAB results to within numerical precision – and, for the parameter values examined here, to four decimal places exactly – and adds Jupyter notebooks, automated figure generation, and a packaging structure suitable for reproducible research.

The main numerical findings are:

- The H^{-1} mix norm decays approximately exponentially under the LTD optimal velocity, consistent with the theoretical predictions of [1], and consistent with the monotonic-decrease property established rigorously in Proposition 1.
- The mixing timescale $\tau(a) = -1/\lambda(a)$ grows with a , confirming that scalars with wider initial support mix more slowly, while the mix norm reached at the stopping time is non-monotone in a , reflecting a trade-off between faster mixing and earlier resolution loss for smaller a .
- The observed power-law scaling $\lambda \sim a^{-1.78}$ deviates moderately from the predicted a^{-1} ; the resolution-sensitivity study of Section 6 shows this exponent is itself resolution-dependent ($a^{-1.61}$ at $N = 32$ vs. $a^{-1.78}$ at $N = 64$), indicating the deviation is at least partly a finite-resolution artefact rather than solely evidence of the greedy strategy’s theoretical sub-optimality.

- The L^p conservation laws provide a robust, physically motivated stopping criterion, and stand in for the explicit dealiasing filtering that a production spectral code would typically apply (Section 3.3): the simulation halts automatically when the grid can no longer resolve the scalar’s fine-scale structures, rather than silently accumulating aliasing error.

Future work could explore:

- Higher resolution ($N = 128$ or $N = 256$), ideally combined with explicit 2/3-rule dealiasing, to separate genuine mixing-rate sub-optimality from finite-resolution truncation effects, and to access the true asymptotic regime before the resolution check intervenes.
- Alternative initial data (e.g. random fields, or the `idata_diag/idata_strip/idata_trigpoly` families already implemented but not swept in Section 5) and their effect on the mixing rate.
- Comparison with non-greedy optimal strategies, e.g. velocities that account for the effect of stirring over a finite time horizon rather than instantaneously, to directly probe the greedy-vs-global optimality question raised in [2, 3].
- A systematic study of the saddle-point degeneracy identified in Section 2.6, including initial data explicitly engineered to approach $Pg \approx 0$.

A Core Source Code

For reference, this appendix reproduces verbatim the two functions of `mixing.py` implementing, respectively, the spectral operator setup of Section 3 and the five-step RHS evaluation of Section 3.2.

A.1 `build_operators`

```

1 def build_operators(N):
2     """
3     Build FFT-based derivative and inverse Laplacian operators on
4     [0,1]^2.
5
6     Parameters
7     -----
8     N: int (power of 2 recommended)
9
10    Returns
11    -----
12    ops: dict with keys
13    N, dx, DEL_X (1xN), DEL_Y (Nx1), LAP_INV (NxN),
14    LAMBDA_INV (NxN), xx (NxN), yy (NxN)
15    """
16    dx = 1.0 / N
17    k = np.arange(N)
18    # Centred wavenumbers: 0, 1, ..., N/2-1, -N/2, ..., -1

```

```

18     k_eff = (k - N * (k > N // 2)).astype(float)
19
20     # Second-derivative operator (Nyquist mode is fine here)
21     kx = (2j * np.pi * k_eff)[np.newaxis, :] # shape (1, N):
        acts on axis-1 (x)
22     ky = (2j * np.pi * k_eff)[:, np.newaxis] # shape (N, 1):
        acts on axis-0 (y)
23     lap = kx**2 + ky**2 # shape (N, N),
        broadcast
24     LAP_INV = np.zeros_like(lap)
25     nz = lap != 0
26     LAP_INV[nz] = 1.0 / lap[nz]
27
28     # H-1 multiplier:  $\lambda^{-1} = \sqrt{-\Delta^{-1}} = 1/(2\pi|k|)$ , zero at  $k=0$ 
29     LAMBDA_INV = np.sqrt(np.where(LAP_INV != 0, -LAP_INV, 0.0))
30
31     # First-derivative operators: zero the Nyquist mode to avoid
        aliasing
32     k_d = k_eff.copy()
33     if N % 2 == 0:
34         k_d[N // 2] = 0.0
35     DEL_X = (2j * np.pi * k_d)[np.newaxis, :]
36     DEL_Y = (2j * np.pi * k_d)[:, np.newaxis]
37
38     x = np.arange(N) * dx
39     xx, yy = np.meshgrid(x, x) # xx[i,j] = x[j], yy[i,j] = x[i]
40
41     return {
42         'N': N, 'dx': dx,
43         'DEL_X': DEL_X, 'DEL_Y': DEL_Y,
44         'LAP_INV': LAP_INV, 'LAMBDA_INV': LAMBDA_INV,
45         'xx': xx, 'yy': yy,
46     }

```

A.2 make_convection_hat

```

1 def make_convection_hat(ops, F=1.0, tol=1e-3):
2     """
3     Build the ODE RHS for the optimal-mixing transport equation.
4
5     The state vector  $y$  in  $\mathbb{R}^{2N^2}$  stores  $[\text{Re}(\theta_{\text{hat}}), \text{Im}(\theta_{\text{hat}})]$ 
6     concatenated.
7
8     Parameters
9     -----
10    ops: dict from build_operators
11    F: enstrophy constraint ( $\|\text{grad}u\|_{L^2} = F$  at each instant)

```

```

12  """tol: threshold for saddle-point warning
13
14  """Returns
15  """-----
16  """callable f(t, y) -> y_dot (real-valued, shape 2N^2)
17  """
18      N = ops['N']
19      n = N * N
20      DEL_X, DEL_Y, LAP_INV = ops['DEL_X'], ops['DEL_Y'], ops['
        LAP_INV']
21
22  def rhs(t, y_real):
23      theta_hat = (y_real[:n] + 1j * y_real[n:]).reshape(N, N)
24
25      # g = theta * grad(Delta^-1 theta)
26      linv_th = LAP_INV * theta_hat
27      theta = np.real(np.fft.ifft2(theta_hat))
28      gx_hat = np.fft.fft2(theta * np.real(np.fft.ifft2(DEL_X
        * linv_th)))
29      gy_hat = np.fft.fft2(theta * np.real(np.fft.ifft2(DEL_Y
        * linv_th)))
30
31      # Leray projection P(g) = g - grad(Delta^-1(div g))
32      div_g = DEL_X * gx_hat + DEL_Y * gy_hat
33      pgx = gx_hat - DEL_X * LAP_INV * div_g
34      pgy = gy_hat - DEL_Y * LAP_INV * div_g
35
36      # v = -Delta^-1 P(g)
37      vx_hat = -LAP_INV * pgx
38      vy_hat = -LAP_INV * pgy
39
40      # ||grad v||_{L2} via Parseval: ||f||_{L2} = ||f_hat||_2
        / N^2
41      norm_v_hat = np.sqrt(
42          np.linalg.norm((DEL_X * vx_hat).ravel())**2 +
43          np.linalg.norm((DEL_X * vy_hat).ravel())**2 +
44          np.linalg.norm((DEL_Y * vx_hat).ravel())**2 +
45          np.linalg.norm((DEL_Y * vy_hat).ravel())**2
46      )
47      norm_v = norm_v_hat / N**2
48
49      # Saddle-point warning (Section 2.6)
50      norm_linv_g = np.linalg.norm(
51          np.stack([LAP_INV * gx_hat, LAP_INV * gy_hat]).ravel
        ()
52      ) / N**2
53      if norm_v < norm_linv_g * tol:
54          print(f'Possible saddle point: t={t:.3f}, '
55                f'||grad v||={norm_v:.3e}, ||Delta^-1 g||={
        norm_linv_g:.3e}')
56

```

```

57     # u = F v / ||grad v||
58     ux = F * np.real(np.fft.ifft2(vx_hat)) / norm_v
59     uy = F * np.real(np.fft.ifft2(vy_hat)) / norm_v
60
61     # d(theta_hat)/dt = -FFT(u . grad theta)
62     d = -np.fft.fft2(
63         ux * np.real(np.fft.ifft2(DEL_X * theta_hat)) +
64         uy * np.real(np.fft.ifft2(DEL_Y * theta_hat))
65     )
66     return np.concatenate([d.real.ravel(), d.imag.ravel()])
67
68     return rhs

```

References

- [1] Z. Lin, J.-L. Thiffeault, and C. R. Doering, Optimal stirring strategies for passive scalar mixing, *Journal of Fluid Mechanics*, 675:465–476, 2011.
- [2] G. Iyer, A. Kiselev, and X. Xu, Lower bounds on the mix norm of passive scalars advected by incompressible eddying flows, *Nonlinearity*, 27(5):973–985, 2014.
- [3] T. D. Drivas, T. M. Elgindi, G. Iyer, and I.-J. Jeong, Anomalous dissipation in passive scalar transport, *Archive for Rational Mechanics and Analysis*, 243(3):1151–1180, 2022.
- [4] L. N. Trefethen, *Spectral Methods in MATLAB*, SIAM, 2000.
- [5] J. P. Boyd, *Chebyshev and Fourier Spectral Methods*, 2nd edition, Dover Publications, 2001.